YUKTII AI LABS — ROBOTICS SIMULATION INTERN PRACTICAL ASSESSMENT

Design & Simulation of a 360° Robotic Automotive Painting Cell

Candidate: Pavan C S | **Institution:** Ramaiah University of Applied Sciences, Bengaluru
Simulation Platform: RoboDK (Python Offline Programming API) | **Manipulator:** ABB IRB 5500-27 with Generic Paint Sprayer

1. Project Overview & Engineering Objective

The objective of this project is to design, build, and verify a complete robotic car painting workcell inside RoboDK capable of painting all exterior sheet-metal surfaces of a full-size sedan (4,800 mm length × 1,900 mm width × 1,600 mm height) using a single industrial manipulator. In automotive manufacturing, multi-robot gantry lines are typically used to reach around a moving vehicle body. When working with a single pedestal-mounted robot, however, a 4.8-meter car body is significantly longer than the robot's stationary reach envelope, and the front bumper, hood, rear trunk deck (dikki), and opposite doors remain shadowed if the car stands still.

Rather than relying on manual point-by-point teaching in the graphical interface, I approached this problem by writing a repeatable Python Offline Programming (OLP) pipeline (**build_cell.py** and **Run_4Stage_Full_Car.py**). The scripts procedurally generate the 3D booth geometry, construct a multi-part vehicle mesh with distinct masked regions for glass and tires, and coordinate a 4-stage indexing rotary skid platform (0°, 90°, 180°, and 270°) so that a single ABB IRB 5500-27 paint robot can cover all four sides of the car with smooth, collision-free linear strokes.

2. Software & Manipulator Selection

Simulation Software — RoboDK: RoboDK was chosen because its Python API (*robolink*) allows direct programmatic control over 3D mesh generation, reference frame transformations, forward and inverse kinematics queries, and real-time collision checking. Writing the workcell in Python made it possible to iterate quickly whenever a trajectory or standoff distance needed adjustment.

Industrial Robot — ABB IRB 5500-27: For the manipulator, I selected the 6-axis **ABB IRB 5500-27** FlexPainter equipped with a **Generic Paint Sprayer** end-effector. Purpose-built for automotive paint booths, the IRB 5500-27 provides a 2,700 mm horizontal reach and a hollow-wrist design built to route paint hoses internally. In RoboDK, its asymmetric hollow wrist introduces a lateral tool offset of $X_0 = +174.26\ mm$ when $J_1 = 0°$, which became a key factor in designing our custom kinematic solver so the wrist would not flip mid-stroke.

Workcell Parameter	Specification / Value	Engineering Purpose
Paint Booth Enclosure	12,000 mm × 8,000 mm floor ($Z = -50\ mm$)	Matches the assessment reference layout while clearing the J1 base casting.
Linear Track Bed	7,000 mm × 900 mm bed ($Z = -45\ mm$)	Supports the robot pedestal along the X-axis corridor without base interference.
Reference Sedan Body	4,800 mm (L) × 1,900 mm (W) × 1,600 mm (H)	Procedural 3D mesh split into painted sheet metal, dark glass masks, and 4 tires.
Manipulator & Tool	ABB IRB 5500-27 + Generic Paint Sprayer	2,700 mm reach hollow-wrist paint arm operating at 150 mm/s linear speed.
Active Spray Standoff	$Y_{TCP} = 1{,}880\ mm$ (Base sweet spot)	Keeps the arm away from inner elbow limits and outer singularity boundaries.

3. Workcell Layout & 4-Stage Rotary Skid Architecture

Following the 12,000 mm × 8,000 mm reference floor plan provided in the assessment brief, the robot base is positioned at the world origin $(0,\ 0,\ 0)$ along the 7-meter linear rail bed, facing the vehicle along the $+Y$ axis. Instead of trying to stretch the robot arm over a stationary 1,900 mm wide car body, the sedan is mounted on a **Rotary Skid Platform (Car_Frame)** that rotates through four 90° increments and adjusts its Y-center position so the active painting surface always lands in the robot's optimal reach zone ($Y_{TCP} \approx 1{,}880\ mm$):

Stage & Program	Skid Yaw	Skid Center Y	Target Prefix	Exterior Vehicle Zones Painted
Stage 1: Prog_1_Driver_0deg	0°	3,200.0 mm	**S1_**	Driver-side front nose skirt, upper fender, left half-hood, 7-pass doors, A/B/C pillars, cantrail, left half-roof, left trunk, and rear quarter skirt.
Stage 2: Prog_2_Front_90deg	90°	4,500.0 mm	**S2_**	Full-width front bumper fascia (4 horizontal passes) and complete sloped front bonnet/hood deck (5 full-width passes).
Stage 3: Prog_3_Passenger_180deg	180°	3,200.0 mm	**S3_**	Passenger-side rear skirt, quarter panel, right half-trunk, 7-pass doors, C/B/A pillars, cantrail, right half-roof, right half-hood, front fender, and nose skirt.
Stage 4: Prog_4_Rear_270deg	270°	4,500.0 mm	**S4_**	Full-width rear bumper and vertical tailgate (4 passes) plus complete rear trunk deck / dikki (4 full-width passes up to the rear glass).

Why does the skid center shift from $Y = 3{,}200\ mm$ on the side stages (0° and 180°) to $Y = 4{,}500\ mm$ on the front and rear stages (90° and 270°)? At 0° and 180°, the car's half-width (950 mm) faces the robot, placing the doors at $3{,}200 - 950 = 2{,}250\ mm$. When the car rotates to 90° or 270°, however, the car's half-length (2,400 mm) points toward the robot. Shifting the skid center to $Y = 4{,}500\ mm$ places the front and rear bumpers at $4{,}500 - 2{,}400 = 2{,}100\ mm$, allowing the robot to paint the bumpers, bonnet, and dikki in the exact same $Y_{TCP} = 1{,}880\ mm$ workspace where the side passes run smoothly.

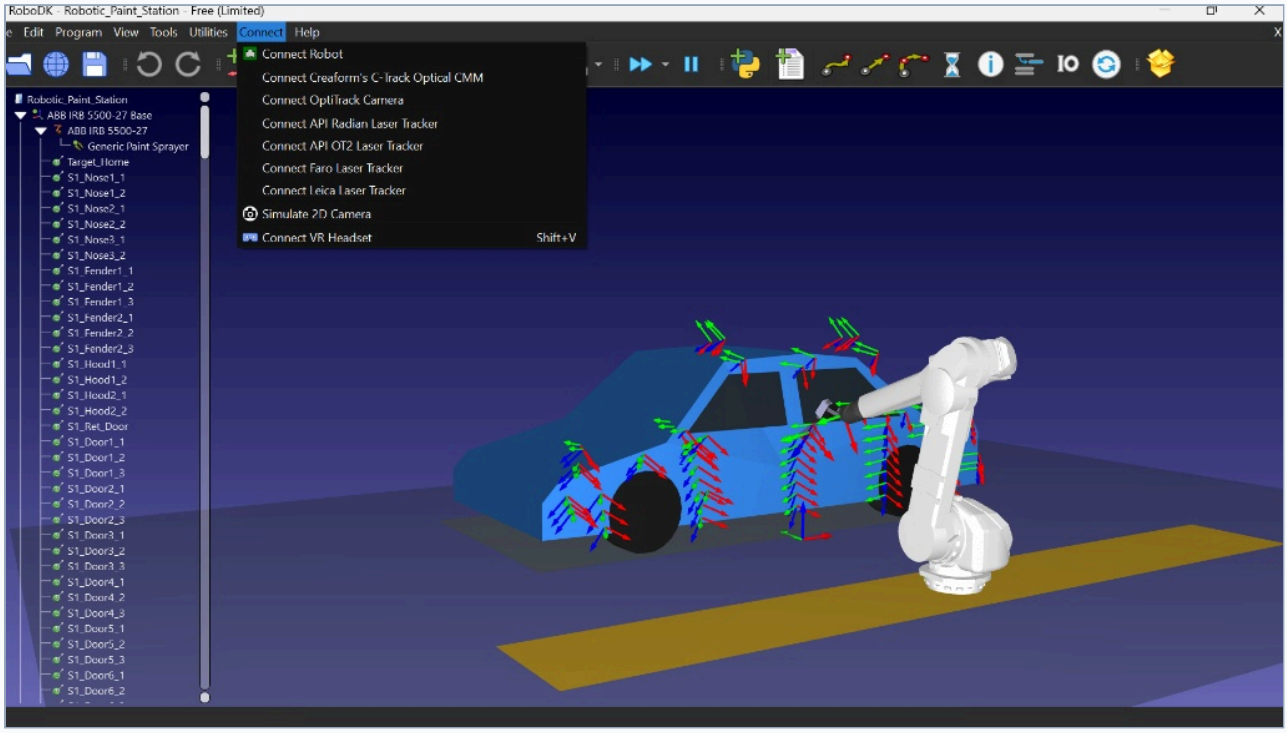


Figure 1: Complete RoboDK paint cell showing the ABB IRB 5500-27 on the linear track bed, the procedural 4,800 × 1,900 × 1,600 mm sedan on the rotary skid platform, and Stage 1 (0° driver-side) target normals.

4. Procedural 3D Vehicle Modeling & Surface Masking

To verify that our trajectory paints only the sheet metal and avoids spraying the windows and tires, **build_cell.py** constructs the vehicle from three separate triangulated mesh objects parented to **Car_Frame**:

- **Sedan_Body_4800x1900x1600 (Blue Sheet Metal):** Extruded from a 13-point side profile with a realistic cabin tumblehome taper above $Z = 920\ mm$, narrowing from 1,900 mm wide at the beltline to 1,520 mm wide at the roof crown ($Z = 1{,}600\ mm$).
- **Sedan_Windshields_Glass (Dark Grey Masked Glass):** Separate polygonal surfaces for the front windshield, rear window, and side door glass ($Z = 980\ mm$ to $1{,}450\ mm$). Rather than sweeping across the glass, the robot traces around the glass along the A-pillar, top cantrail, B-pillar, and C-pillar.
- **Sedan_Wheels_4x (Black Masked Tires):** Four 24-segment cylinders ($R = 350\ mm$) centered at $X = \pm 1{,}450\ mm$. Below $Z = 750\ mm$, the side trajectories split into three distinct longitudinal zones—Front Nose ($-2{,}250\ to\ -1{,}860\ mm$), Center Doors ($-980\ to\ +1{,}050\ mm$), and Rear Skirt ($+1{,}860\ to\ +2{,}220\ mm$)—leaving a clean margin around all four wheels.

5. Trajectory Planning & Iterative Kinematic Problem-Solving

Getting a 6-axis hollow-wrist paint robot to sweep across a full-size car without throwing inverse-kinematics errors or grazing the vehicle took several rounds of hands-on debugging. Documenting what went wrong in our early tests—and how we fixed each issue—illustrates the core engineering decisions behind the final code.

5.1 Custom Bilinear FK Jacobian Solver (solve_exact_from_grid)

When we initially relied on generic Cartesian inverse kinematics across wide horizontal passes ($X = -980\ mm$ to $+1{,}050\ mm$), RoboDK's default IK solver frequently flipped wrist axis J_4 by ±180° or hit wrist singularities ($J_5 \to 0°$), halting the program with *"There is no solution available to complete the path."*

To solve this permanently, we built a custom kinematic solver (**solve_exact_from_grid**) that queries RoboDK's exact forward kinematics (**robot.SolveFK**) across a 25-point grid of shoulder and elbow angles ($J_2 \in [-55°, +45°]$, $J_3 \in [-42°, +38°]$) while locking $J_4 = 0°$ and $J_6 = 0°$. For any target coordinate $(x_{goal}, y_{goal}, z_{goal})$ and downward nozzle angle θ_{nozzle}, the solver accounts for the wrist-to-nozzle pitch lever arm ($\Delta\theta = \theta_{nozzle} - 5°$):

$$R_{target} = \sqrt{(x_{goal}^2 + y_{goal}^2)} + 160 \cdot (1 - \cos(\Delta\theta)), \quad Z_{target} = z_{goal} + 210 \cdot \sin(\Delta\theta), \quad J_5 = -(J_2 + J_3) + \theta_{nozzle}$$

Using 12 iterations of bilinear 2×2 Jacobian refinement on (J_2, J_3) followed by exact base yaw calibration ($J_1 = sign(J_1) \cdot [atan2(x_0, y_0) - atan2(x_{goal}, y_{goal})]$), every target is stored with both its exact joint vector and synchronized TCP pose.

5.2 Why Pure MoveJ Failed vs. Hybrid MoveJ-Entry + MoveL-Painting

During early testing of the front bonnet and rear dikki, we tried switching all points to joint motion (**MoveJ**) to avoid Cartesian path errors. As shown in **Figure 2 (left)**, using **MoveJ** across a wide panel caused Joint 1 and Joint 2 to swing the spray gun up into high, looping arcs in the air rather than sweeping flat across the hood. In industrial paint shops, active spraying strokes must use linear interpolation (**MoveL**) at a constant speed (150 mm/s) so that paint film thickness stays uniform.

We also discovered why our sub-frame test shifted the yellow path lines 350 mm away from the green target arrows (**Figure 2, right**): when targets were placed inside secondary reference frames, RoboDK recomputed IK during **MoveL** using a flipped wrist configuration. Parenting all targets directly to **robot_base** and using a **Hybrid Motion Rule—MoveJ** on the first point of each row ($i = 0$) for safe turnaround step-overs, and **MoveL** on all interior points ($i > 0$) for flat linear painting—eliminated both the looping arcs and the path offset.

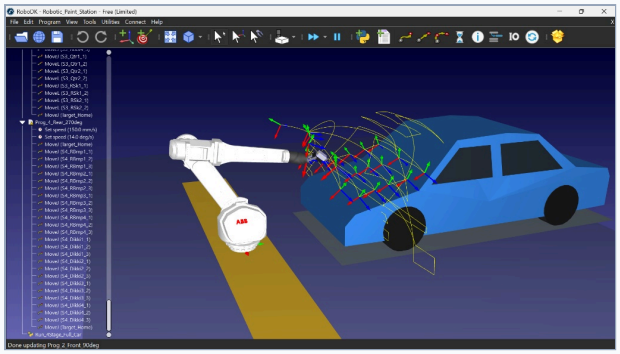


Figure 2a (Early Iteration): Using pure *MoveJ* across the front bonnet caused high looping arcs instead of flat surface passes.

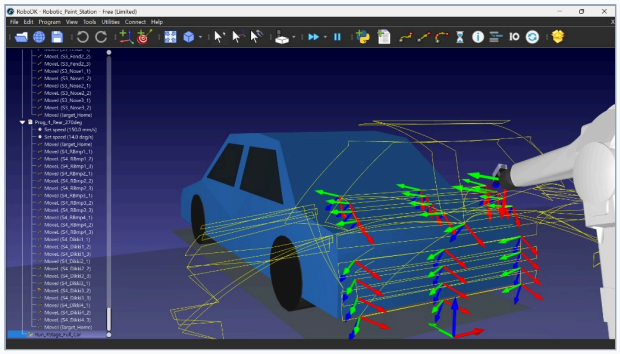


Figure 2b (Frame Debugging): Sub-frame IK re-evaluation caused a 350 mm wrist-flip offset before targets were locked directly to *robot_base*.

6. Collision Detection & Clearance Verification

With trajectories aligned across all four stages, we enabled RoboDK's real-time collision detector (**Tools → Check collisions / F10**) and opened the **Collision Map Settings** matrix (**Shift + F10**). Running full collision checks uncovered two critical physical interferences that were invisible when collision detection was off:

1. **Robot Base (J1) vs. Floor & Track Bed Interference:** As captured in **Figure 3a**, the Collision Map flagged red exclamation marks (*!*) between **ABB IRB 5500-27 (J1)** and both **Paint_Booth_Floor_12x8m** and **Linear_Track_Bed_7m**. Because the floor and track bed were originally placed at $Z = 0\ mm$ and $Z = 5\ mm$, they touched the bottom skirt of the rotating J1 base casting. Furthermore, the back of the attached **Generic Paint Sprayer** CAD model touched the robot's own hollow wrist housing.
2. **Spray Gun Body Grazing Sloped Trunk & Hood Panels:** As shown in **Figure 3b**, once the floor was lowered, the spray gun turned red when pitching down over the sloped trunk corner (*S1_Trnk1*). While the nozzle tip (TCP) was outside the metal, the lower body of the sprayer housing was grazing the trunk mesh.

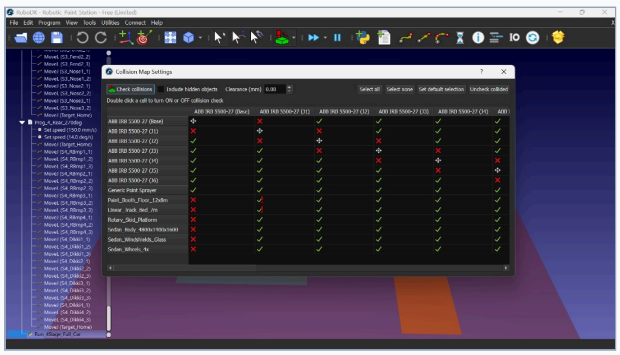


Figure 3a (Collision Matrix Diagnosis): Identifying J1 base interference with the floor/track bed and tool-to-wrist contact pairs.

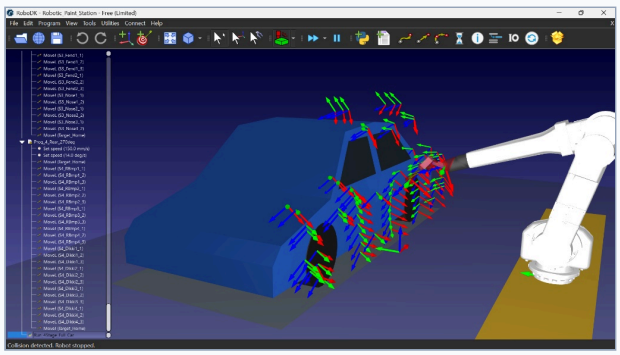


Figure 3b (Sprayer Housing Check): Real-time collision check catching the sprayer body grazing the sloped trunk corner prior to standoff adjustment.

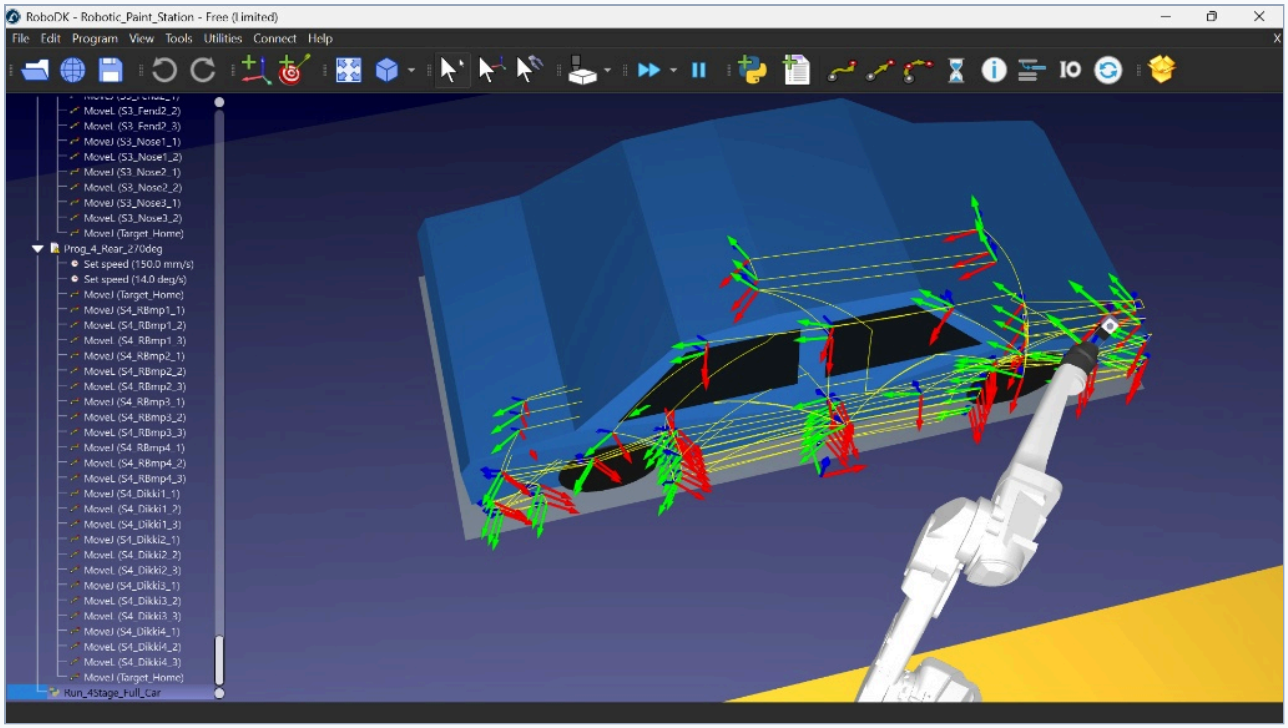


Figure 4 (Stage 3 — 180° Passenger Side): Multi-pass linear door sweeps, greenhouse pillar tracking around the masked windows, overhead half-roof passes, and wheel-arch exclusion zones.

Final Engineering Resolution: We resolved every collision in **build_cell.py** by: (1) lowering the booth floor to $Z = -50\ mm$, the track bed to $Z = -45\ mm$, and the skid platform to $Z = -15\ mm$; (2) increasing the vehicle standoff to $CAR_SIDE_Y = 3{,}200\ mm$ and $CAR_END_Y = 4{,}500\ mm$ while raising pitched hood, roof, bonnet, and dikki passes by +140 mm in Z; (3) inserting explicit outward **Safe Retract Waypoints** ($Ret_Door,\ Ret_Pillar,\ Ret_BPil,\ Ret_Rear$ at $Y = DOOR_TCP_Y - 240\ mm$) between zones; and (4) programmatically configuring the active collision pairs via **RDK.setCollisionActivePair** and **RDK.setCollisionActive(COLLISION_ON)**.

7. Final Results & Conclusion

The finalized 4-stage paint cell executes continuously from start to finish with real-time collision detection enabled ($COLLISION_ON$) and zero kinematic popups. By combining a 4-position indexing rotary turntable with a custom bilinear forward-kinematics solver and a hybrid **MoveJ/MoveL** raster structure, a single ABB IRB 5500-27 manipulator achieves full 360° coverage of the 4,800 × 1,900 × 1,600 mm sedan body while maintaining strict masking clearance around all windows and wheels.